

Task 05 — Build Basic Java Microservices System

Step 1 — Create Spring Boot projects

Go to:

[Spring Initializr](#)

Create **two Spring Boot projects**.

Project 1

Name: user-service

Artifact: user-service

Package: com.example.userservice

Java: 17

Packaging: Jar

Dependencies:

Spring Web

Spring Data JPA

MySQL Driver

Validation

Spring Boot Actuator (optional)

Project 2

Name: order-service

Artifact: order-service

Package: com.example.orderservice

Java: 17

Packaging: Jar

Dependencies:

Spring Web

Spring Data JPA

MySQL Driver

Validation

Download both ZIP files and extract them.

Open both projects in **IntelliJ IDEA**.

Step 2 — Create User Service

In IntelliJ:

user-service

└─ src

└─ main

└─ java

└─ com.example.userservice

Create these packages:

controller

service

repository

entity

exception

Final structure:

user-service

└─ src/main/java/com/example/userservice

└─ controller

└─ UserController.java

```
└─ service
    └─ UserService.java
└─ repository
    └─ UserRepository.java
└─ entity
    └─ User.java
└─ UserServiceApplication.java
```

Step 3 — Create Order Service

Create similar packages in order-service.

order-service

```
└─ controller
    └─ OrderController.java
└─ service
    └─ OrderService.java
└─ repository
    └─ OrderRepository.java
└─ entity
    └─ Order.java
└─ OrderServiceApplication.java
```

Step 4 — Configure databases

Separate databases.

You can use **one MySQL server with two separate databases**.

Open MySQL Workbench and run:

```
CREATE DATABASE user_db;
CREATE DATABASE order_db;
```

Now you have:

MySQL

├─ user_db

└─ order_db

This demonstrates database separation between services.

Step 5 — Create entities

User Entity

In:

user-service

→ entity

→ User.java

Create:

```
package com.example.userservice.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    public User() {
    }

    public User(String name, String email) {
        this.name = name;
    }
}
```

```
        this.email = email;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Order Entity

In:

order-service

→ entity

→ Order.java

Create:

```
package com.example.orderservice.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "orders")

public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long userId;

    private String product;

    private Double amount;

    public Order() {
    }

    public Long getId() {
        return id;
    }

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public String getProduct() {
        return product;
    }
}
```

```

public void setProduct(String product) {
    this.product = product;
}

public Double getAmount() {
    return amount;
}

public void setAmount(Double amount) {
    this.amount = amount;
}
}

```

Notice that we store only:

userId

inside Order.

We don't create a JPA relationship to the User entity because **User Service owns the user data**.

Step 6 — Create repositories

UserRepository

Create:

user-service

→ repository

→ UserRepository.java

```

package com.example.userservice.repository;

import com.example.userservice.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
}

```

OrderRepository

Create:

order-service

→ repository

→ OrderRepository.java

```
package com.example.orderservice.repository;

import com.example.orderservice.entity.Order;
import org.springframework.data.jpa.repository.JpaRepository;

public interface OrderRepository extends JpaRepository<Order, Long> {

}
```

Step 7 — Create services

UserService

```
package com.example.userservice.service;

import com.example.userservice.entity.User;
import com.example.userservice.repository.UserRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service

public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {

        this.userRepository = userRepository;

    }

    public User createUser(User user) {

        return userRepository.save(user);

    }

}
```

```

public List<User> getAllUsers() {
    return userRepository.findAll();
}

public User getUserById(Long id) {
    return userRepository.findById(id)
        .orElseThrow(() ->
            new RuntimeException("User not found"));
}
}

```

OrderService

```

package com.example.orderservice.service;

import com.example.orderservice.entity.Order;
import com.example.orderservice.repository.OrderRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service

public class OrderService {

    private final OrderRepository orderRepository;

    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public Order createOrder(Order order) {
        return orderRepository.save(order);
    }

    public List<Order> getAllOrders() {
        return orderRepository.findAll();
    }
}

```

```

    }

    public Order getOrderById(Long id) {
        return orderRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Order not found"));
    }
}

```

Step 8 — Create REST Controllers

UserController

```

package com.example.userservice.controller;

import com.example.userservice.entity.User;
import com.example.userservice.service.UserService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/users")

public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }

    @GetMapping

```

```

        public List<User> getAllUsers() {
            return userService.getAllUsers();
        }

        @GetMapping("/{id}")
        public User getUser(@PathVariable Long id) {
            return userService.getUserById(id);
        }
    }
}

```

OrderController

```

package com.example.orderservice.controller;

import com.example.orderservice.entity.Order;
import com.example.orderservice.service.OrderService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/orders")

public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public Order createOrder(@RequestBody Order order) {
        return orderService.createOrder(order);
    }

    @GetMapping

```

```

public List<Order> getAllOrders() {
    return orderService.getAllOrders();
}

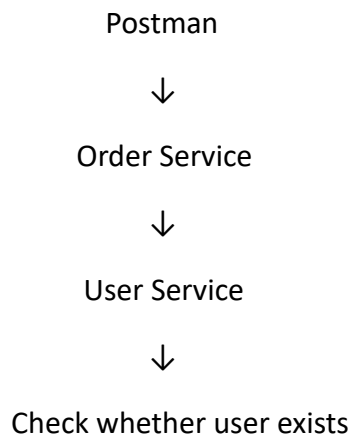
@GetMapping("/{id}")
public Order getOrder(@PathVariable Long id) {
    return orderService.getOrderById(id);
}
}

```

Step 9 — Implement inter-service communication

The Order Service should communicate with User Service.

For example:



You can use Spring's RestClient.

Add this to Order Service:

```

package com.example.orderservice.service;

import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

@Service
public class UserClient {

    private final RestClient restClient;

```

```

public UserClient(RestClient.Builder builder) {
    this.restClient = builder
        .baseUrl("http://localhost:8081")
        .build();
}

public boolean userExists(Long userId) {
    try {
        restClient.get()
            .uri("/users/{id}", userId)
            .retrieve()
            .toBodilessEntity();

        return true;
    } catch (Exception e) {
        return false;
    }
}
}

```

Then modify OrderService:

```

private final OrderRepository orderRepository;
private final UserClient userClient;

public OrderService(OrderRepository orderRepository,
    UserClient userClient) {

    this.orderRepository = orderRepository;
    this.userClient = userClient;
}

public Order createOrder(Order order) {

```

```
if (!userClient.userExists(order.getUserId())) {  
    throw new RuntimeException("User does not exist");  
}  
  
return orderRepository.save(order);  
}
```

Now actual:

Service-to-Service Communication.

Step 10 — Add validation

For User:

```
import jakarta.validation.constraints.Email;  
import jakarta.validation.constraints.NotBlank;  
  
@NotBlank  
private String name;  
  
@NotBlank  
@Email  
private String email;
```

Then controller:

```
@PostMapping  
public User createUser(@Valid @RequestBody User user) {  
    return userService.createUser(user);  
}
```

Make sure you have:

```
import jakarta.validation.Valid;
```

For Order:

```
@NotNull  
private Long userId;  
  
@NotBlank  
private String product;  
  
@NotNull  
@Positive  
private Double amount;
```

This allows you to demonstrate validation.

For example, this should fail:

```
{  
  "userId": null,  
  "product": "",  
  "amount": -10  
}
```

Step 11 — Add exception handling**Create:**

exception

└─ ResourceNotFoundException.java

└─ GlobalExceptionHandler.java

Example:

```
package com.example.orderservice.exception;  
  
public class ResourceNotFoundException extends RuntimeException {  
  public ResourceNotFoundException(String message) {  
    super(message);  
  }  
}
```

```
}
```

Global handler:

```
package com.example.orderservice.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> handleNotFound(
        ResourceNotFoundException ex) {
        return Map.of("error", ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public Map<String, String> handleException(Exception ex) {
        return Map.of("error", ex.getMessage());
    }
}
```

This satisfies:

Error handling.

Step 12 — Test all APIs using Postman

First run User Service

In IntelliJ:

UserServiceApplication

Run it.

It should run on:

`http://localhost:8081`

Set in:

`application.properties`

`server.port=8081`

`spring.datasource.url=jdbc:mysql://localhost:3306/user_db`

`spring.datasource.username=root`

`spring.datasource.password=YOUR_PASSWORD`

`spring.jpa.hibernate.ddl-auto=update`

`spring.jpa.show-sql=true`

Run Order Service

Set:

`server.port=8082`

`spring.datasource.url=jdbc:mysql://localhost:3306/order_db`

`spring.datasource.username=root`

`spring.datasource.password=YOUR_PASSWORD`

`spring.jpa.hibernate.ddl-auto=update`

`spring.jpa.show-sql=true`

Run: `OrderServiceApplication`

It should run on: `http://localhost:8082`

Postman testing

Open **Postman**.

Test 1 — Create User

POST

http://localhost:8081/users

Body → raw → JSON:

```
{  
  "name": "Madhava",  
  "email": "madhava@example.com"  
}
```

Expected:

```
{  
  "id": 1,  
  "name": "Madhava",  
  "email": "madhava@example.com"  
}
```

Test 2 — Get Users

GET http://localhost:8081/users

Expected:

```
[  
  {  
    "id": 1,  
    "name": "Madhava",  
    "email": "madhava@example.com"  
  }  
]
```

Test 3 — Get User by ID

GET

http://localhost:8081/users/1

Expected:

```
{  
  "id": 1,  
  "name": "Madhava",  
  "email": "madhava@example.com"  
}
```

Test 4 — Create Order

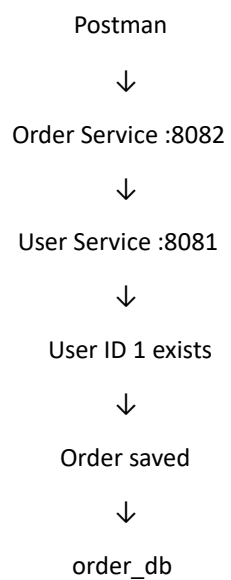
POST

http://localhost:8082/orders

Body:

```
{  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 55000  
}
```

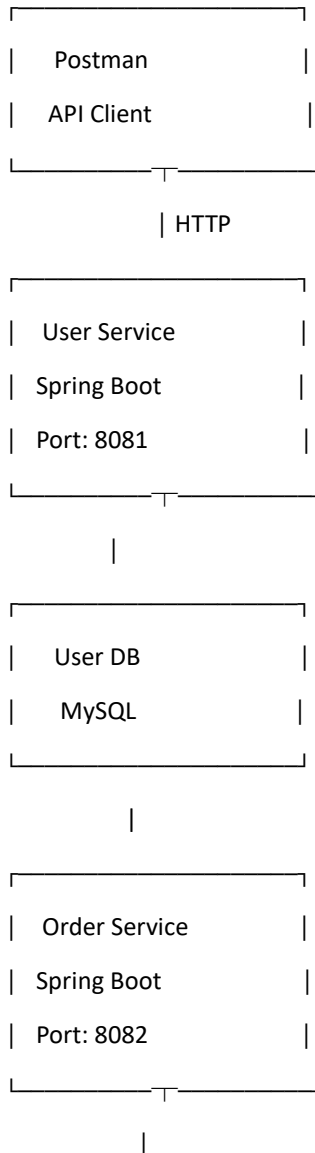
Flow:

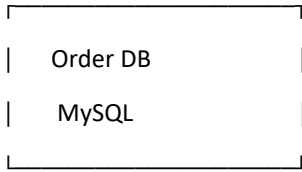


Expected:

```
{  
  "id": 1,  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 55000  
}
```

Step 14. Create architecture diagram





Order Service — HTTP —> User Service